

PostgreSQL - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

POSTGRESQL — MASTER NOTES	6
A Complete Reference Book for Interviews, Production Work & System Understanding	6
Written for someone who has never installed or touched PostgreSQL before	6
 TOPIC 1: INSTALLATION	 6
1. Introduction	6
2. Installing PostgreSQL on Windows — Step by Step	6
Step 1: Download the installer	6
Step 2: Run the installer	6
Step 3: Finish installation	6
3. Verifying the Installation	7
4. Connecting to PostgreSQL for the First Time — Two Ways	7
WAY 1: psql — The Command-Line Tool (what we'll use throughout this document)	7
WAY 2: pgAdmin 4 — The Graphical Tool (beginner-friendly, visual)	7
5. Real Life Analogy	7
6. Common Mistakes	8
7. Frequently Asked Interview Questions	8
 TOPIC 2: DATABASE MANAGEMENT	 8
1. Introduction	8
2. Opening psql and the Essential "Meta-Commands" (\ commands)	8
3. Creating Your First Database and Table — Full Walkthrough	10
4. Creating a Database WITHOUT psql — Using createdb (Terminal Shortcut)	10
5. Deleting a Database	11
6. Doing the Exact Same Thing in pgAdmin (Visual Alternative)	11
7. Real Life Analogy	11
8. Edge Cases	11
9. Common Mistakes	11
10. Practice Questions	12
11. Frequently Asked Interview Questions	12

TOPIC 3: ROLES & PERMISSIONS	12
1. Introduction	13
2. Creating Roles — Full Commands	13
3. Granting and Revoking Privileges	13
4. Key Role Attributes	13
5. Real Life Analogy	14
6. Best Practices	14
7. Frequently Asked Interview Questions	14
 TOPIC 4: DATA TYPES (POSTGRESQL-SPECIFIC)	 14
1. Introduction	14
2. Key PostgreSQL-Specific Data Types	15
3. TEXT vs VARCHAR(n) — A PostgreSQL-Specific Nuance	15
4. NUMERIC vs FLOAT/DOUBLE PRECISION — Critical for Money	15
5. Real Life Analogy	15
6. Practice Questions	15
7. Frequently Asked Interview Questions	15
 TOPIC 5: INDEXING (POSTGRESQL-SPECIFIC)	 16
1. Introduction	16
2. PostgreSQL's Index Types	16
3. Which Index Type to Use — Decision Table	17
4. Real Life Analogy	17
5. Frequently Asked Interview Questions	17
 TOPIC 6: JSONB	 17
1. Introduction	17
2. JSON vs JSONB — Which to Use	18
3. Code Example — Creating and Querying JSONB	18
4. -> vs ->> — A Frequently Confused Pair	19
5. Real Life Analogy	19
6. Edge Cases	19
7. Practice Questions	19

8. Frequently Asked Interview Questions	19
---	----

TOPIC 7: ARRAYS **20**

1. Introduction	20
2. Code Example — Creating and Querying Arrays	20
3. When to Use an Array vs a Separate Child Table	21
4. Real Life Analogy	21
5. Edge Cases	21
6. Practice Questions	21
7. Frequently Asked Interview Questions	22

TOPIC 8: FULL TEXT SEARCH **22**

1. Introduction	22
2. Why LIKE Isn't Enough	22
3. Full Text Search — tsvector and tsquery	22
4. Making It Fast — A Dedicated tsvector Column + GIN Index	23
5. Real Life Analogy	23
6. Frequently Asked Interview Questions	23

TOPIC 9: PERFORMANCE TUNING **23**

1. Introduction	23
2. EXPLAIN ANALYZE — PostgreSQL's Key Diagnostic Tool	24
3. Key Performance Tools/Commands	24
4. Why VACUUM Matters (A PostgreSQL-Specific Concept)	24
5. Real Life Analogy	24
6. Common Mistakes	25
7. Frequently Asked Interview Questions	25

TOPIC 10: BACKUP & RESTORE **25**

1. Introduction	25
2. pg_dump — Backing Up a Database	25
3. psql/pg_restore — Restoring From a Backup	26
4. Plain SQL vs Custom Format — Comparison Table	26

5. Real Life Analogy	26
6. Edge Cases	26
7. Common Mistakes	26
8. Best Practices	27
9. Frequently Asked Interview Questions	27
10. Revision Notes (Entire PostgreSQL Section Consolidated)	28
11. Homework (Covers Entire PostgreSQL Section)	28
FINAL SUMMARY — POSTGRESQL COMPLETE	28

POSTGRESQL — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

Written for someone who has never installed or touched PostgreSQL before

TOPIC 1: INSTALLATION

1. Introduction

What is it? PostgreSQL (often called "Postgres") is a free, open-source, powerful relational database management system (RDBMS) — the actual SOFTWARE you install and run on your computer/server that stores and manages your data, and understands the SQL you learned in the previous section.

Why PostgreSQL specifically? It's one of the most popular production databases in the world (used by Instagram, Spotify, Reddit), is fully free and open-source, and adds powerful extra features beyond standard SQL (JSONB, arrays, full-text search — all covered later in this section) that many companies rely on heavily.

2. Installing PostgreSQL on Windows — Step by Step

Step 1: Download the installer

Go to <https://www.postgresql.org/download/windows/> in your browser, click "Download the installer" (this takes you to the EDB installer page), and download the latest version (e.g., PostgreSQL 16 or 17) for Windows x86-64.

Step 2: Run the installer

Double-click the downloaded .exe file. The installation wizard will ask you for:

- **Installation directory:** Leave the default (usually C:\Program Files\PostgreSQL\17).
- **Components to install:** Keep ALL checked — this includes:
 - **PostgreSQL Server** (the actual database engine)
 - **pgAdmin 4** (a free graphical tool for managing your database visually)
 - **Command Line Tools** (psql and others)
 - **Stack Builder** (optional, for extra add-ons — not needed for learning)
- **Data directory:** Leave the default.
- **Password:** You will be asked to set a password for the default superuser account, called postgres. **WRITE THIS DOWN — you will need it constantly.**
- **Port number:** Leave the default, 5432 (the standard PostgreSQL port).
- **Locale:** Leave the default.

Step 3: Finish installation

Click through to finish. PostgreSQL now runs as a **Windows Service** in the background — meaning it starts automatically every time you turn on your computer, and you don't need to manually "start" it each time.

3. Verifying the Installation

Open a fresh PowerShell/terminal window and run:

```
psql --version
```

If this prints something like `psql (PostgreSQL) 17.0`, the command-line tool is installed and on your PATH correctly.

4. Connecting to PostgreSQL for the First Time — Two Ways

WAY 1: `psql` — The Command-Line Tool (what we'll use throughout this document)

```
psql -U postgres
```

- `-U postgres` means "connect as the user named `postgres`" (the default superuser created during installation).
- You will be PROMPTED for the password you set during installation.
- Once connected, your prompt changes to `postgres=#` — you are now INSIDE the PostgreSQL command-line client, ready to type SQL commands and special `psql` commands (covered in the next topic).

If `psql` isn't found as a command, add PostgreSQL's `bin` folder to your Windows PATH:

```
$env:Path += ";C:\Program Files\PostgreSQL\17\bin"
```

(To make this PERMANENT, search "Edit the system environment variables" in Windows, then add `C:\Program Files\PostgreSQL\17\bin` to the PATH variable there.)

WAY 2: pgAdmin 4 — The Graphical Tool (beginner-friendly, visual)

- 1 Open the Start Menu and search for "**pgAdmin 4**" — it was installed alongside PostgreSQL.
- 2 It opens in your web browser (pgAdmin runs as a local web application).
- 3 You'll be asked to set a MASTER PASSWORD the first time (this is just for pgAdmin itself, different from the PostgreSQL `postgres` user password).
- 4 In the left sidebar, expand **Servers** → **PostgreSQL 17** — it will ask for the `postgres` user's password (the one from installation).
- 5 Once connected, you can see **Databases**, expand them, and browse **Schemas** → **public** → **Tables** visually — right-click any of these for options like "Create Table," "View Data," etc.

Recommendation for this course: We will show `psql` (command-line) commands throughout, since they're universal, scriptable, and exactly what you'll use in real production/server environments — but everything shown here can ALSO be done by clicking around in pgAdmin if you prefer a visual approach. Many real database administrators use BOTH depending on the task.

5. Real Life Analogy

- **Restaurant:** Installing PostgreSQL is like installing an entire commercial kitchen setup in your restaurant building — `psql` is like a direct intercom line to the kitchen where you speak precise instructions (typed SQL commands); pgAdmin is like a visual control panel with buttons and screens showing you exactly what's happening in the kitchen, letting you point-and-click instead.

6. Common Mistakes

- Forgetting the password set during installation — if truly lost, it requires editing PostgreSQL's authentication configuration file (`pg_hba.conf`) to temporarily allow passwordless local access, then resetting it — an advanced recovery step, best avoided by writing the password down clearly the first time.
- Trying to run `psql` from a terminal without it being on the PATH, getting a "command not found" error.
- Confusing pgAdmin's own MASTER PASSWORD (for pgAdmin itself) with PostgreSQL's `postgres` USER password (for the actual database) — these are two completely separate passwords.

7. Frequently Asked Interview Questions

- 1 **What is PostgreSQL?** A free, open-source, powerful relational database management system.
- 2 **What is `psql`?** PostgreSQL's official command-line client tool for connecting to and interacting with a PostgreSQL database.
- 3 **What is pgAdmin?** A free, official graphical (browser-based) administration tool for managing PostgreSQL databases visually.
- 4 **What is the default PostgreSQL port?** 5432.
- 5 **What is the default PostgreSQL superuser called?** `postgres`.

TOPIC 2: DATABASE MANAGEMENT

1. Introduction

What is it? This topic covers the actual day-to-day commands for creating, listing, switching between, and deleting databases — plus creating and inspecting TABLES inside them, entirely from the `psql` command line (or the equivalent pgAdmin clicks) — everything you need to actually START WORKING with a real local database.

2. Opening `psql` and the Essential "Meta-Commands" (`\` commands)

```
psql -U postgres
```

Once connected (prompt shows `postgres=#`), these SPECIAL commands (NOT plain SQL — they start with a backslash, and are `psql`-specific, not standard SQL) are essential:

Command	Purpose
\l	LIST all databases on this PostgreSQL server
\c database_name	CONNECT to (switch into) a specific database
\dt	LIST all tables in the CURRENTLY connected database
\d table_name	DESCRIBE a specific table's columns, types, and constraints (like DESCRIBE in MySQL)
\du	LIST all database users/roles
\q	QUIT psql
\?	Show help for all psql meta-commands

3. Creating Your First Database and Table — Full Walkthrough

```
-- Step 1: While connected as 'postgres', CREATE a new database for our practice
CREATE DATABASE company_db;

-- Step 2: SWITCH into that new database (this is a psql META-command, note the backslash)
\c company_db

-- You should now see the prompt change to: company_db=#
-- This confirms you're now "inside" company_db, not the default postgres database

-- Step 3: CREATE a table (standard SQL, exactly like what you learned in the SQL topic)
CREATE TABLE departments (
dept_id SERIAL PRIMARY KEY, -- SERIAL = PostgreSQL's auto-incrementing integer type
dept_name VARCHAR(50) NOT NULL,
location VARCHAR(50)
);

CREATE TABLE employees (
emp_id SERIAL PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
dept_id INT REFERENCES departments(dept_id),
salary DECIMAL(10,2),
hire_date DATE
);

-- Step 4: VERIFY the tables were created
\dt
-- List of relations
-- Schema | Name | Type | Owner
-- -----+-----+-----+-----
-- public | departments | table | postgres
-- public | employees | table | postgres

-- Step 5: INSERT some sample data
INSERT INTO departments (dept_name, location) VALUES
('Engineering', 'Bangalore'),
('Sales', 'Mumbai');

INSERT INTO employees (first_name, last_name, dept_id, salary, hire_date) VALUES
('Alice', 'Johnson', 1, 75000, '2023-01-15'),
('Bob', 'Smith', 2, 60000, '2022-06-01');

-- Step 6: QUERY it (standard SQL, exactly as learned before)
SELECT * FROM employees;

-- Step 7: Inspect a table's exact structure
\d employees
-- Table "public.employees"
-- Column | Type | Collation | Nullable | Default
--
-----+-----+-----+-----+-----
-- emp_id | integer | | not null | nextval('employees_emp_id_seq'::regclass)
-- first_name | character varying(50) | | not null |
-- ...
```

4. Creating a Database WITHOUT `psql` — Using `createdb` (Terminal Shortcut)

```
createdb -U postgres company_db2
```

This is a convenient shortcut command-line tool that does the same thing as `CREATE DATABASE`, without needing to first connect via `psql`.

5. Deleting a Database

```
DROP DATABASE company_db2; -- run this WHILE CONNECTED TO A DIFFERENT DATABASE, e.g., postgres
```

Critical rule: You CANNOT drop a database while you are currently CONNECTED to it — first `\c postgres` (switch to a different database), THEN run `DROP DATABASE company_db2;`.

6. Doing the Exact Same Thing in pgAdmin (Visual Alternative)

- 1 Right-click **Databases** in the left sidebar → **Create** → **Database...** → type `company_db` → Save.
- 2 Expand `company_db` → **Schemas** → **public** → right-click **Tables** → **Create** → **Table...** → fill in the name and add columns via the dialog's "Columns" tab.
- 3 Right-click any table → **View/Edit Data** → **All Rows** to see its data visually, or **Query Tool** (a button in the toolbar) to type and run raw SQL, exactly like `psql`.

7. Real Life Analogy

- **Library:** `CREATE DATABASE` is like establishing an entirely new, separate library building. `\c database_name` is like walking INTO that specific library building (you can only be inside ONE library location at a time). `CREATE TABLE` is like installing a new labeled bookshelf inside that library. `\dt` is like asking "what shelves exist in the library I'm currently standing in?"

8. Edge Cases

Case	Result
Running <code>\dt</code> right after connecting, before creating any tables	Shows "Did not find any relations"
Trying <code>DROP DATABASE</code> while connected to that SAME database	Error: "cannot drop the currently open database"
Creating a table with the SAME name as an existing one	Error: "relation already exists"
Forgetting <code>\c database_name</code> after <code>CREATE DATABASE</code>	Your <code>CREATE TABLE</code> commands run against whichever database you were ALREADY connected to (often the default <code>postgres</code> database), NOT your new one — a very common beginner mistake

9. Common Mistakes

- Creating a database, then FORGETTING to `\c` into it before creating tables — ending up with tables in the wrong (default) database.
- Confusing `\c` (a `psql` meta-command, no semicolon needed) with standard SQL commands (which DO need a semicolon `;` at the end).
- Trying to `DROP DATABASE` while still connected to it.

10. Practice Questions

Q1 (Easy): Create a new database called `school_db`, connect to it, and verify no tables exist yet.

```
CREATE DATABASE school_db;
\c school_db
\dt
```

Q2 (Medium): Inside `school_db`, create a `students` table with `student_id` (auto-incrementing primary key), `name`, and `grade`, then verify its structure using a meta-command.

```
CREATE TABLE students (
  student_id SERIAL PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  grade INT
);
\d students
```

Q3 (Hard, interview-style): Explain, step by step, exactly what happens if you run `CREATE DATABASE test_db;` while connected to `postgres`, then immediately run `CREATE TABLE users (id INT);` WITHOUT running `\c test_db` first.

Answer: The `CREATE TABLE` command executes successfully, but it creates the `users` table inside the `postgres` database (the one you're STILL connected to), NOT inside the newly created `test_db`. Connecting to `test_db` and running `\dt` there would show NO tables — you'd have to `\c postgres` and run `\dt` to find the misplaced `users` table. The fix is to ALWAYS run `\c test_db` immediately after creating a new database, before creating any tables inside it.

11. Frequently Asked Interview Questions

- 1 What command lists all databases in `psql`? `\l`.
- 2 What command switches your current connection to a different database? `\c database_name`.
- 3 What command lists all tables in the currently connected database? `\dt`.
- 4 Why can't you drop a database you're currently connected to? PostgreSQL requires no active connections/sessions to a database before it can be safely dropped, to avoid corrupting an in-use database.
- 5 What does `SERIAL` mean as a column type? PostgreSQL's shorthand for an auto-incrementing integer column, internally implemented using a sequence.

TOPIC 3: ROLES & PERMISSIONS

1. Introduction

What is it? In PostgreSQL, "roles" are the unified concept for BOTH users and groups (unlike some databases that treat them separately) — a role can log in (acting as a "user"), or just group permissions together (acting as a "group"), or both. Permissions (`GRANT/REVOKE`, standard SQL DCL from the previous section) control exactly what each role can do.

2. Creating Roles — Full Commands

```
-- Create a role that CAN log in (a "user"), with a password
CREATE ROLE app_user WITH LOGIN PASSWORD 'securepassword123';

-- Create a role WITHOUT login (a pure "group" role, for organizing permissions)
CREATE ROLE readonly_group;

-- Give app_user membership in the readonly_group role
GRANT readonly_group TO app_user;

-- A shorthand that creates a login-capable role in one step (identical to above with LOGIN):
CREATE USER analyst WITH PASSWORD 'analystpass456';
```

`CREATE USER` is actually just a convenient ALIAS for `CREATE ROLE ... WITH LOGIN` — in PostgreSQL, "users" and "roles" are the SAME underlying concept.

3. Granting and Revoking Privileges

```
-- Grant SELECT-only access to a specific table
GRANT SELECT ON employees TO readonly_group;

-- Grant full read/write access
GRANT SELECT, INSERT, UPDATE, DELETE ON employees TO app_user;

-- Grant access to ALL tables in a schema (convenient for broad access)
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_group;

-- Revoke a specific privilege
REVOKE INSERT ON employees FROM app_user;

-- View all current roles
\du
```

4. Key Role Attributes

Attribute	Meaning
LOGIN	Can be used to actually connect/authenticate to the database
SUPERUSER	Bypasses ALL permission checks — extremely powerful, use sparingly
CREATEDB	Can create new databases
CREATEROLE	Can create other roles
PASSWORD 'xxx'	Sets the login password
VALID UNTIL 'date'	Sets an expiration date for the role's password

5. Real Life Analogy

- **Bank:** A role is like a bank staff ID badge — some badges (roles) can physically SWIPE INTO the building (LOGIN), some are just LABELS grouping a set of permissions together ("Teller Group," "Manager Group") that get assigned to actual staff badges. A SUPERUSER role is like the bank's master key that opens literally everything — reserved for the most trusted, senior administrator only.

6. Best Practices

- NEVER use the postgres superuser account for regular application connections — create a dedicated, LIMITED-privilege role (e.g., app_user) for your actual application to connect with.
- Follow the "principle of least privilege" — grant ONLY the specific permissions a role genuinely needs, nothing more.
- Use group roles (like readonly_group) to organize permissions cleanly, then assign individual login roles as MEMBERS of those groups, rather than granting the same permissions repeatedly to each individual user.

7. Frequently Asked Interview Questions

- 1 **What's the difference between a "role" and a "user" in PostgreSQL?** They're the SAME underlying concept — CREATE USER is simply a convenient alias for CREATE ROLE ... WITH LOGIN.
- 2 **What does the LOGIN attribute control?** Whether a role can actually be used to authenticate/connect to the database, versus being purely a permission-grouping role.
- 3 **Why is it a bad practice for an application to connect using the postgres superuser account?** It violates the principle of least privilege — a compromised or buggy application connection would then have UNRESTRICTED access to everything, instead of only what it actually needs.
- 4 **How do you grant a role membership in a group role?** GRANT group_role_name TO member_role_name;.

TOPIC 4: DATA TYPES (POSTGRESQL-SPECIFIC)

1. Introduction

What is it? While PostgreSQL supports all standard SQL data types (INT, VARCHAR, DATE, etc. — covered in the SQL section), it ALSO provides many powerful, PostgreSQL-SPECIFIC data types that go well beyond the standard SQL specification — some of which (JSONB, arrays) get their own dedicated topics later in this document.

2. Key PostgreSQL-Specific Data Types

```
CREATE TABLE example_types (  
  id SERIAL PRIMARY KEY,  
  small_num SMALLINT, -- 2 bytes, -32,768 to 32,767  
  regular_num INTEGER, -- 4 bytes (standard)  
  big_num BIGINT, -- 8 bytes, for very large numbers  
  precise_money NUMERIC(10,2), -- exact, arbitrary-precision decimal (like Java's BigDecimal)  
  is_active BOOLEAN DEFAULT TRUE, -- true/false/NULL  
  description TEXT, -- UNLIMITED length string (unlike VARCHAR's optional limit)  
  unique_id UUID DEFAULT gen_random_uuid(), -- universally unique identifier  
  tags TEXT[], -- ARRAY of text values (full topic later!)  
  metadata JSONB, -- binary JSON (full topic later!)  
  created_at TIMESTAMP DEFAULT NOW(), -- date + time, no timezone  
  created_at_tz TIMESTAMPTZ DEFAULT NOW(), -- date + time, WITH timezone awareness  
  ip_address INET, -- stores IP addresses (IPv4 or IPv6) natively  
  price_range INT4RANGE -- a RANGE type - stores a range of integers!  
);
```

3. TEXT vs VARCHAR(n) — A PostgreSQL-Specific Nuance

Unlike some other databases, PostgreSQL's TEXT and VARCHAR have **virtually IDENTICAL performance** internally — VARCHAR(n) only adds a LENGTH CHECK on top of the same underlying storage TEXT uses. In many other databases, choosing between fixed vs variable-length types is a genuine performance decision; in PostgreSQL, it's much more about whether you WANT a length constraint enforced, not a performance one.

4. NUMERIC vs FLOAT/DOUBLE PRECISION — Critical for Money

```
-- NEVER use FLOAT/DOUBLE PRECISION for money - binary floating point causes rounding errors  
price_wrong FLOAT, -- BAD for currency (same reason as Java's double, see Core Java Topic 6)  
  
-- ALWAYS use NUMERIC/DECIMAL for exact currency values  
price_correct NUMERIC(10,2) -- GOOD - exact decimal representation, no rounding surprises
```

5. Real Life Analogy

- **Bank:** Using FLOAT for account balances is like using an imprecise measuring cup for currency — small rounding errors compound over millions of transactions. NUMERIC is like using an exact, certified scale that guarantees precision down to the last cent, every single time.

6. Practice Questions

Q1 (Easy): Create a products table with an exact-precision price column and a TEXT description.

```
CREATE TABLE products (  
  product_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  price NUMERIC(10,2) NOT NULL,  
  description TEXT  
);
```

Q2 (Medium): Add a UUID column with an auto-generated default value to an existing orders table.

```
ALTER TABLE orders ADD COLUMN order_uuid UUID DEFAULT gen_random_uuid();
```

7. Frequently Asked Interview Questions

- 1 **Why should you use NUMERIC instead of FLOAT for currency values?** FLOAT/DOUBLE PRECISION use binary floating-point representation, which cannot exactly represent many decimal fractions, causing rounding errors; NUMERIC provides exact, arbitrary-precision decimal arithmetic.
- 2 **Is there a meaningful performance difference between TEXT and VARCHAR(n) in PostgreSQL?** No — they use the same underlying storage; VARCHAR(n) just adds an explicit length constraint check.
- 3 **What does the UUID type store, and how can you auto-generate one?** A universally unique identifier; use DEFAULT gen_random_uuid() (requires the pgcrypto extension in older versions, built-in since PostgreSQL 13+).

TOPIC 5: INDEXING (POSTGRESQL-SPECIFIC)

1. Introduction

(General indexing theory — B-Trees, the leftmost-prefix rule, read/write trade-offs — is fully covered in the SQL section's Indexes topic. This topic focuses specifically on PostgreSQL's MULTIPLE index TYPES, since Postgres supports far more than just the standard B-Tree.)

2. PostgreSQL's Index Types

```
-- B-TREE (the DEFAULT, used for most equality/range queries - covered in the SQL section)
CREATE INDEX idx_employees_dept ON employees(dept_id);

-- HASH (optimized ONLY for exact equality lookups, NOT ranges)
CREATE INDEX idx_employees_email_hash ON employees USING HASH (email);

-- GIN (Generalized Inverted Index) - for JSONB, ARRAYS, and full-text search (covered soon!)
CREATE INDEX idx_employees_metadata_gin ON employees USING GIN (metadata);

-- GiST (Generalized Search Tree) - for geometric data, full-text search, range types
CREATE INDEX idx_price_range_gist ON products USING GIST (price_range);

-- BRIN (Block Range Index) - extremely lightweight, for HUGE tables with naturally ORDERED data
CREATE INDEX idx_events_created_brin ON events USING BRIN (created_at);
```


3. Which Index Type to Use — Decision Table

Index Type	Best For
B-Tree (default)	Equality AND range queries (=, <, >, BETWEEN, ORDER BY) — the correct default choice for MOST columns
Hash	ONLY exact equality (=) lookups — rarely chosen over B-Tree in practice, since B-Tree handles equality well too PLUS ranges
GIN	JSONB columns, ARRAY columns, full-text search vectors — anything where a single column value contains MULTIPLE searchable "items"
GiST	Geometric/spatial data, range types, full-text search (an alternative to GIN with different trade-offs)
BRIN	Extremely large tables where data is naturally stored in the SAME ORDER it's queried (e.g., a created_at timestamp column on an append-only log table) — very small index size, much less precise than B-Tree

4. Real Life Analogy

- **Library:** A B-Tree index is the library's standard alphabetical card catalog. A GIN index is like a specialized catalog that lets you search by ANY keyword INSIDE a book's full content (not just its title) — appropriate for JSONB/array/full-text scenarios where you're searching WITHIN a complex value, not just comparing it directly.

5. Frequently Asked Interview Questions

- 1 **What's PostgreSQL's default index type?** B-Tree.
- 2 **When would you use a GIN index instead of a B-Tree?** For JSONB, ARRAY, or full-text search columns, where you need to search WITHIN a complex value rather than compare the whole value directly.
- 3 **What's special about a BRIN index?** It's extremely small/lightweight, ideal for huge tables where data is naturally stored in the same order it's typically queried (like a timestamp on an append-only table), trading precision for size.

TOPIC 6: JSONB

1. Introduction

What is it? JSONB ("JSON Binary") is PostgreSQL's native, INDEXED, binary-optimized storage format for JSON data directly inside a relational column — letting you combine the flexibility of semi-structured/schema-less JSON documents with the reliability, transactions, and query power of a full relational database.

Why it matters: Sometimes data doesn't fit neatly into rigid, predefined columns (e.g., a product's varying custom attributes, a user's flexible preferences). `JSONB` lets you store such flexible data WITHOUT needing an entirely separate NoSQL database, while still being able to query, index, and filter on specific fields WITHIN that JSON.

2. `JSON` vs `JSONB` — Which to Use

Aspect	<code>JSON</code>	<code>JSONB</code>
Storage	Stores the EXACT text you inserted, as-is	Stores a parsed, BINARY-optimized representation
Insert speed	Slightly faster (no parsing on insert)	Slightly slower (parses on insert)
Query/read speed	Slower (must re-parse text on every query)	MUCH faster (already parsed/optimized)
Supports indexing (GIN)?	No	Yes
Preserves key order/whitespace/duplicate keys?	Yes	No — normalized on storage
Recommendation	Rarely used in practice	Use JSONB for virtually everything — the standard, correct choice

3. Code Example — Creating and Querying `JSONB`

```
CREATE TABLE products (  
  product_id SERIAL PRIMARY KEY,  
  name VARCHAR(100),  
  attributes JSONB  
);  
  
INSERT INTO products (name, attributes) VALUES  
(  
  'Laptop', '{"brand": "Dell", "ram_gb": 16, "colors": ["silver", "black"]}',  
  'Phone', '{"brand": "Samsung", "ram_gb": 8, "colors": ["blue"]}'  
);  
  
-- ACCESS a top-level key (returns JSONB)  
SELECT name, attributes -> 'brand' AS brand FROM products;  
  
-- ACCESS a top-level key AS TEXT (returns plain text, not JSONB)  
SELECT name, attributes ->> 'brand' AS brand FROM products;  
  
-- FILTER based on a JSONB field's value  
SELECT name FROM products WHERE attributes ->> 'brand' = 'Dell';  
  
-- FILTER using the CONTAINS operator (@>) - "does this JSONB contain this smaller JSONB?"  
SELECT name FROM products WHERE attributes @> '{"brand": "Dell"}';  
  
-- CHECK if a key EXISTS  
SELECT name FROM products WHERE attributes ? 'ram_gb';  
  
-- UPDATE a specific JSONB key's value  
UPDATE products SET attributes = jsonb_set(attributes, '{"ram_gb"}', '32') WHERE name = 'Laptop';  
  
-- CREATE a GIN index for fast JSONB querying  
CREATE INDEX idx_products_attributes ON products USING GIN (attributes);
```

4. `->` vs `->>` — A Frequently Confused Pair

Operator	Returns
<code>-></code>	The value as JSONB (keeps quotes/JSON typing — useful for further JSONB operations or nested access)
<code>->></code>	The value as plain TEXT (strips quotes — useful for comparisons, display, or casting to another SQL type)

5. Real Life Analogy

- **Restaurant:** A relational table's fixed columns are like a STANDARDIZED order form with fixed fields (table number, order time). JSONB is like an attached "special notes" section on that same form, where a customer can jot down ANY flexible custom request ("extra spicy," "no onions, allergic to nuts") — the SAME order form (row) combines both the rigid, standardized fields AND the flexible, free-form notes.

6. Edge Cases

Case	Result
Accessing a non-existent key with <code>-></code> or <code>->></code>	Returns SQL NULL, no error
Storing duplicate keys in JSONB	Only the LAST occurrence of a duplicate key is kept — JSONB normalizes/deduplicates on storage (unlike plain JSON)
Comparing JSONB values with <code>=</code>	Works, but compares the ENTIRE JSON structure exactly; use <code>@></code> for "contains" style partial matching instead

7. Practice Questions

Q1 (Easy): Find all products where the `brand` attribute is "Samsung".

```
SELECT name FROM products WHERE attributes ->> 'brand' = 'Samsung';
```

Q2 (Medium): Find all products that have "black" somewhere in their `colors` array (inside the JSONB).

```
SELECT name FROM products WHERE attributes -> 'colors' ? 'black';
```

Q3 (Hard, interview-style): Explain why you would add a GIN index on a JSONB column before running frequent queries against it in production, and what would happen WITHOUT one.

Answer: Without a GIN index, every JSONB query (`->>`, `@>`, `?`) requires PostgreSQL to scan and parse EVERY row's JSONB value sequentially (a full table scan) — extremely slow on large tables. A GIN index builds an inverted index over the JSONB structure's keys/values, letting these operators use fast index lookups instead, similar in spirit to how a B-Tree speeds up equality/range lookups on regular columns.

8. Frequently Asked Interview Questions

- 1 **What's the difference between JSON and JSONB?** JSON stores the exact original text; JSONB stores a parsed, binary-optimized, indexable representation — JSONB is faster to query and is the standard recommended choice.
- 2 **What's the difference between `->` and `->>`?** `->` returns the value as JSONB; `->>` returns it as plain TEXT.

- 3 **What does the @> operator do?** Checks whether a JSONB value CONTAINS a specified smaller JSONB structure — useful for flexible partial matching.
 - 4 **What index type should you use for a frequently-queried JSONB column?** GIN.
-
-

TOPIC 7: ARRAYS

1. Introduction

What is it? PostgreSQL uniquely (among mainstream relational databases) allows a SINGLE column to store an ARRAY of values (multiple values of the same type) directly — instead of always needing a separate child table for one-to-many relationships.

2. Code Example — Creating and Querying Arrays

```
CREATE TABLE employees (  
  emp_id SERIAL PRIMARY KEY,  
  name VARCHAR(50),  
  skills TEXT[], -- an ARRAY of text values  
  scores INTEGER[] -- an ARRAY of integers  
);  
  
INSERT INTO employees (name, skills, scores) VALUES  
(  
  'Alice', ARRAY['Java', 'SQL', 'Python'], ARRAY[90, 85, 95]),  
(  
  'Bob', ARRAY['JavaScript', 'React'], ARRAY[80, 75]);  
  
-- ACCESS a specific array element (PostgreSQL arrays are 1-INDEXED, not 0-indexed!)  
SELECT name, skills[1] AS first_skill FROM employees;  
  
-- CHECK if a value EXISTS anywhere in the array (the ANY operator)  
SELECT name FROM employees WHERE 'SQL' = ANY(skills);  
  
-- CHECK if the array CONTAINS all specified values (the @> "contains" operator)  
SELECT name FROM employees WHERE skills @> ARRAY['Java', 'SQL'];  
  
-- GET the LENGTH of an array  
SELECT name, array_length(skills, 1) AS num_skills FROM employees;  
  
-- EXPAND (unnest) an array into SEPARATE ROWS, one per element  
SELECT name, unnest(skills) AS skill FROM employees;  
-- name | skill  
-- -----+-----  
-- Alice | Java  
-- Alice | SQL  
-- Alice | Python  
-- Bob   | JavaScript  
-- Bob   | React  
  
-- CREATE a GIN index for fast array containment/overlap queries  
CREATE INDEX idx_employees_skills ON employees USING GIN (skills);
```

3. When to Use an Array vs a Separate Child Table

Scenario	Recommendation
Simple, small, rarely-queried-individually list of values (e.g., a few tags)	An array CAN be a reasonable, convenient choice
The related values need their OWN additional attributes, need to be individually referenced/joined elsewhere, or the list can grow very large	Use a proper separate child table with a foreign key (standard relational/normalized design, see the Normalization topic) — the array approach breaks down and becomes awkward

Interview-relevant nuance: Arrays are a genuinely useful PostgreSQL-specific convenience feature, but overusing them where a proper normalized child table would be more appropriate is a common design mistake — they work best for simple, denormalized "tag"-style data, not core relational structures.

4. Real Life Analogy

- **Restaurant:** An array column is like a single order ticket with a handwritten LIST of extra toppings jotted in one spot ("extra cheese, olives, mushrooms") — convenient for a simple list, but if you needed to track the PRICE, SUPPLIER, and STOCK LEVEL of each individual topping separately, you'd need a proper separate "toppings" table instead, linked back to the order.

5. Edge Cases

Case	Result
Accessing <code>skills[0]</code> (index 0)	Returns <code>NULL</code> — PostgreSQL arrays are 1-INDEXED, so index 0 is always "before the start"
Accessing an out-of-range index (e.g., <code>skills[100]</code> on a 3-element array)	Returns <code>NULL</code> , no error
An empty array <code>ARRAY[]::TEXT[]</code>	Valid, <code>array_length()</code> returns <code>NULL</code> for an empty array (a subtle gotcha — not 0!)

6. Practice Questions

Q1 (Easy): Find all employees who know "Python".

```
SELECT name FROM employees WHERE 'Python' = ANY(skills);
```

Q2 (Medium): List every employee alongside each of their individual skills as SEPARATE rows.

```
SELECT name, unnest(skills) AS skill FROM employees;
```

Q3 (Hard, interview-style): Explain when using a `TEXT[]` array column for "skills" would be a POOR design choice compared to a separate `employee_skills` join table.

Answer: If skills need their OWN additional data (e.g., a proficiency LEVEL, a certification DATE per skill), or if you need to efficiently query/aggregate "how many employees know each skill" across the WHOLE table with proper indexing and joins, a normalized `employee_skills(emp_id, skill_name, proficiency_level)` table is far more appropriate — the array approach works well only for simple, flat lists with no per-element metadata.

7. Frequently Asked Interview Questions

- 1 **Are PostgreSQL arrays 0-indexed or 1-indexed?** 1-indexed — `array[1]` is the FIRST element.
 - 2 **What does the `@>` operator check for arrays?** Whether the array CONTAINS all the specified values.
 - 3 **What does `unnest()` do?** Expands an array column into SEPARATE ROWS, one per element — useful for treating array contents as if they were normal relational rows.
 - 4 **When should you avoid using an array column?** When the related values need their own additional attributes, or need to be efficiently queried/joined independently — a proper normalized child table is more appropriate in those cases.
-

TOPIC 8: FULL TEXT SEARCH

1. Introduction

What is it? PostgreSQL's built-in Full Text Search lets you search TEXT CONTENT intelligently — matching WORDS regardless of exact case, punctuation, or even different grammatical forms of the same word (e.g., "running" matching a search for "run") — far more powerful than a basic `LIKE '%word%'` pattern match.

2. Why `LIKE` Isn't Enough

```
-- LIKE - basic substring matching, CANNOT use a standard index efficiently, no "word" awareness
SELECT * FROM articles WHERE content LIKE '%running%';
-- Won't match "runs" or "ran" even though they're related words; also generally requires a
-- full table scan since standard B-Tree indexes can't efficiently support a LEADING wildcard search
```

3. Full Text Search — `tsvector` and `tsquery`

```
CREATE TABLE articles (
  article_id SERIAL PRIMARY KEY,
  title VARCHAR(200),
  content TEXT
);

INSERT INTO articles (title, content) VALUES
('Learning Java', 'Java programming is fun. Developers love running Java applications.'),
('Cooking Basics', 'Learn how to cook pasta and other simple recipes.');
```

-- Convert text into a SEARCHABLE, WORD-NORMALIZED format (tsvector), and search with a tsquery:

```
SELECT title FROM articles
WHERE to_tsvector('english', content) @@ to_tsquery('english', 'run');
```

-- MATCHES "Learning Java" - because to_tsvector normalizes "running" down to its root form "run"!

-- Search for MULTIPLE words (both must appear)

```
SELECT title FROM articles
WHERE to_tsvector('english', content) @@ to_tsquery('english', 'java & applications');
```

-- Search for EITHER word

```
SELECT title FROM articles
WHERE to_tsvector('english', content) @@ to_tsquery('english', 'pasta | java');
```

4. Making It Fast — A Dedicated `tsvector` Column + GIN Index

```
-- Add a dedicated column storing the PRE-COMPUTED tsvector (avoids recomputing it on every query)
ALTER TABLE articles ADD COLUMN content_tsv TSVECTOR;
UPDATE articles SET content_tsv = to_tsvector('english', content);

-- Index it for FAST full-text search
CREATE INDEX idx_articles_content_tsv ON articles USING GIN (content_tsv);

-- Now queries against content_tsv use the index efficiently:
SELECT title FROM articles WHERE content_tsv @@ to_tsquery('english', 'run');
```

In real production systems, you'd typically use a **TRIGGER** (see the SQL section's Triggers topic) to automatically keep `content_tsv` up to date whenever `content` changes, rather than manually running `UPDATE` every time.

5. Real Life Analogy

- **Library:** A `LIKE '%word%'` search is like flipping through every single page of every book looking for an exact character sequence. Full Text Search with a `tsvector` + GIN index is like having a smart librarian's card catalog that ALREADY understands "running," "runs," and "ran" are all related to the ROOT WORD "run," and can instantly point you to every relevant book.

6. Frequently Asked Interview Questions

- 1 **Why is PostgreSQL's Full Text Search better than a basic `LIKE '%word%'` query for text search?** It understands word ROOT FORMS (stemming — "running" matches "run"), can use a GIN index for fast lookups, and supports proper multi-word boolean queries (AND/OR) — `LIKE` does none of this and generally can't use a standard index efficiently for leading-wildcard searches.
- 2 **What are `tsvector` and `tsquery`?** `tsvector` is a normalized, searchable representation of a text document; `tsquery` is a normalized search query, matched against a `tsvector` using the `@@` operator.
- 3 **Why would you add a dedicated `tsvector` column instead of computing `to_tsvector()` on every query?** To avoid re-computing the (relatively expensive) text normalization on EVERY single query — precomputing it once and indexing it with GIN makes searches dramatically faster.

TOPIC 9: PERFORMANCE TUNING

1. Introduction

What is it? Performance tuning in PostgreSQL involves understanding and using the tools PostgreSQL provides to diagnose SLOW queries and configure the server for your actual workload — building directly on the general Query Optimization concepts from the SQL section, with PostgreSQL-SPECIFIC tools.

2. `EXPLAIN ANALYZE` — PostgreSQL's Key Diagnostic Tool

```
EXPLAIN ANALYZE
SELECT * FROM employees WHERE dept_id = 1;
```

```
Seq Scan on employees (cost=0.00..15.50 rows=2 width=64) (actual time=0.012..0.045 rows=2 loops=1)
Filter: (dept_id = 1)
Planning Time: 0.089 ms
Execution Time: 0.067 ms
```

- `EXPLAIN` alone shows the ESTIMATED plan (without actually running the query).
- `EXPLAIN ANALYZE` ACTUALLY RUNS the query and shows the REAL execution time and row counts alongside the estimates — critical for spotting when the planner's ESTIMATE is wildly wrong compared to REALITY (a common sign that table statistics need updating via `ANALYZE`).
- Seq Scan means a FULL TABLE SCAN occurred (potentially slow on large tables) — if you expected an index to be used here, this is your signal something's wrong (missing index, or the planner chose not to use an existing one).

3. Key Performance Tools/Commands

```
-- Update the planner's internal STATISTICS about table contents (helps it make better decisions)
ANALYZE employees;

-- Reclaim storage space from deleted/updated rows, and update statistics
VACUUM ANALYZE employees;

-- See currently RUNNING queries (useful for diagnosing a "stuck"/slow production system live)
SELECT pid, query, state, query_start FROM pg_stat_activity WHERE state = 'active';

-- Find your SLOWEST queries over time (requires the pg_stat_statements extension)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT query, calls, total_exec_time, mean_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

4. Why `VACUUM` Matters (A PostgreSQL-Specific Concept)

PostgreSQL uses a technique called **MVCC (Multi-Version Concurrency Control)** — when you `UPDATE` or `DELETE` a row, the OLD version isn't immediately physically removed (it's kept temporarily so other concurrent transactions can still see a consistent snapshot). `VACUUM` is the process that RECLAIMS this leftover "dead" space, preventing table bloat over time. Modern PostgreSQL runs **autovacuum** automatically in the background, but understanding it matters for diagnosing unexpectedly large table sizes.

5. Real Life Analogy

- **Restaurant:** `EXPLAIN ANALYZE` is like a manager reviewing exactly HOW a specific order was prepared (which stations were used, how long each step ACTUALLY took) versus how it was ESTIMATED to go. `VACUUM` is like the restaurant's regular deep-cleaning routine, clearing out old, no-longer-needed leftover materials that pile up from constant food preparation (updates/deletes), keeping the kitchen running efficiently.

6. Common Mistakes

- Disabling `autovacuum` without understanding the consequences, leading to significant table bloat and degraded performance over time.
- Only looking at `EXPLAIN`'s `ESTIMATES` without running `EXPLAIN ANALYZE` to see the `ACTUAL` execution behavior — estimates can be significantly wrong if table statistics are stale.
- Not running `ANALYZE` after a large bulk data load, leaving the query planner with outdated statistics that lead to poor query plans.

7. Frequently Asked Interview Questions

- 1 **What's the difference between `EXPLAIN` and `EXPLAIN ANALYZE`?** `EXPLAIN` shows the `ESTIMATED` query plan without running the query; `EXPLAIN ANALYZE` **ACTUALLY EXECUTES** the query and shows real timing/row-count data alongside the estimates.
 - 2 **What does `VACUUM` do, and why is it necessary in PostgreSQL specifically?** Reclaims storage space left behind by PostgreSQL's MVCC mechanism (old row versions kept temporarily for concurrent transaction consistency) — without it, tables can grow bloated over time.
 - 3 **What does `ANALYZE` (without `VACUUM`) do?** Updates the query planner's internal statistics about table contents, helping it choose better execution plans.
 - 4 **What extension helps you find your slowest queries in production?** `pg_stat_statements`.
-

TOPIC 10: BACKUP & RESTORE

1. Introduction

What is it? Backing up a PostgreSQL database means saving a complete copy of its data/structure to a file, so it can be `RESTORED` later in case of data loss, corruption, or migration to a new server — an absolutely essential, non-optional practice for any real production database.

2. `pg_dump` — Backing Up a Database

```
# Back up a SINGLE database to a plain SQL text file
pg_dump -U postgres -d company_db -f company_db_backup.sql

# Back up in a COMPRESSED, custom format (recommended for larger databases - required for
# selective/parallel restore later)
pg_dump -U postgres -d company_db -F c -f company_db_backup.dump

# Back up ALL databases on the server at once
pg_dumpall -U postgres -f all_databases_backup.sql
```

3. `psql/pg_restore` — Restoring From a Backup

```
# Restore a PLAIN SQL backup (created without -F c) into a NEW, already-created empty database
createdb -U postgres company_db_restored
psql -U postgres -d company_db_restored -f company_db_backup.sql

# Restore a CUSTOM-FORMAT backup (created with -F c) using pg_restore instead
createdb -U postgres company_db_restored
pg_restore -U postgres -d company_db_restored company_db_backup.dump
```

4. Plain SQL vs Custom Format — Comparison Table

Aspect	Plain SQL (<code>sql</code> , default)	Custom Format (<code>F c</code> , <code>dump</code>)
Human-readable?	Yes — it's literally a text file of SQL commands	No — a compressed binary format
Restore tool	<code>psql -f file.sql</code>	<code>pg_restore</code>
Selective restore (only specific tables)?	Difficult (would need manual editing of the SQL file)	Yes — <code>pg_restore</code> supports restoring only specific tables/schemas
Compression	No (unless you compress the file separately)	Yes — built-in
Recommended for	Small databases, or when you want to inspect/edit the backup manually	Larger production databases

5. Real Life Analogy

- **Bank:** `pg_dump` is like the bank creating a complete, certified photocopy of every single record in a vault at a specific moment in time. `pg_restore/psql -f` is like using that photocopy to entirely REBUILD the vault's contents from scratch at a new location, exactly as they were at the moment the backup was taken.

6. Edge Cases

Case	Result
Restoring a plain SQL backup into a database that ALREADY has conflicting tables/data	Will likely produce errors for <code>CREATE TABLE</code> /duplicate key conflicts — restore into a genuinely EMPTY database
Forgetting to create the target database before restoring	<code>psql/pg_restore</code> will fail — the target database must exist first (create it via <code>createdb</code> or <code>CREATE DATABASE</code>)

7. Common Mistakes

- NOT taking regular backups at all — a genuinely common, catastrophic real-world mistake (hardware failure, accidental `DROP TABLE`, or a bad migration can permanently destroy data with no backup to recover from).
- Taking backups but NEVER actually testing a restore — a backup you've never verified you can restore from is not a reliable safety net.

- Storing backups on the SAME physical server/disk as the live database — if that server/disk fails entirely, the backup is lost too; backups should be stored SEPARATELY (different disk, different server, or cloud storage).

8. Best Practices

- Schedule AUTOMATED, REGULAR backups (not manual, easy-to-forget ones) for any real production database.
- Store backups in a SEPARATE location from the live database server.
- Periodically TEST restoring from a backup to verify it actually works — an untested backup strategy is not a real safety net.

9. Frequently Asked Interview Questions

- 1 **What tool creates a PostgreSQL database backup?** `pg_dump` (for a single database) or `pg_dumpall` (for all databases on a server).
- 2 **What's the difference between plain SQL and custom-format backups?** Plain SQL is a human-readable text file restored via `psql -f`; custom format is a compressed binary format restored via `pg_restore`, supporting selective/parallel restore.
- 3 **Why should backups be stored in a separate location from the live database server?** So that a failure of the primary server/disk (hardware failure, corruption) doesn't also destroy the backup meant to recover from exactly that scenario.
- 4 **Why is it important to periodically TEST a restore, not just take backups?** A backup that has never been successfully restored is unverified — there could be corruption, missing data, or process errors that only become apparent during an actual restore attempt, ideally discovered during a test, not a real emergency.

10. Revision Notes (Entire PostgreSQL Section Consolidated)

POSTGRESQL – CHEAT SHEET

=====

INSTALL (Windows): download from postgresql.org -> installer sets up Server + pgAdmin + psql -> remember the 'postgres' user's password!

CONNECT: psql -U postgres (or use pgAdmin's browser-based GUI)

PSQL META-COMMANDS (backslash, NOT standard SQL):

\l list databases \c dbname switch database

\dt list tables \d tablename describe a table

\du list roles/users \q quit

ROLES: CREATE ROLE / CREATE USER (= ROLE + LOGIN), GRANT/REVOKE privileges, NEVER use 'postgres' superuser for app connections

DATA TYPES: SERIAL (auto-increment), NUMERIC (exact, use for money!), TEXT (same perf as VARCHAR), UUID, JSONB, ARRAYS, TIMESTAMPTZ

INDEXES: B-Tree (default, equality+range), GIN (JSONB/arrays/full-text), GiST (geometric/ranges), BRIN (huge, naturally-ordered tables)

JSONB: -> returns JSONB, ->> returns TEXT, @> "contains", ? "key exists" ALWAYS prefer JSONB over plain JSON; index with GIN

ARRAYS: 1-INDEXED (not 0!), ANY() for membership, @> for contains, unnest() to expand into rows

FULL TEXT SEARCH: to_tsvector() + to_tsquery() + @@ operator, understands word roots ("running" matches "run"), index the tsvector column with GIN

PERFORMANCE: EXPLAIN (estimate) vs EXPLAIN ANALYZE (actual execution), VACUUM (reclaims MVCC dead space), ANALYZE (updates planner statistics), pg_stat_statements (find slow queries)

BACKUP/RESTORE: pg_dump (backup) + psql -f / pg_restore (restore), custom format (-F c) for large DBs/selective restore, ALWAYS test restores, NEVER store backups only on the same server

11. Homework (Covers Entire PostgreSQL Section)

- 1 Install PostgreSQL on your own machine (if not already done), connect via psql, create a database called practice_db, and create + populate the employees/departments tables from Topic 2.
- 2 Add a JSONB column to your employees table storing flexible "preferences" data, insert a few rows with different JSON structures, and write 3 queries using ->, ->>, and @>.
- 3 Run EXPLAIN ANALYZE on a simple SELECT before and after adding an index on the filtered column, and compare the execution plans.
- 4 Take a pg_dump backup of practice_db, drop the database entirely, then restore it from your backup file, verifying all data is intact afterward.

FINAL SUMMARY — POSTGRESQL COMPLETE

This concludes the **PostgreSQL** section — 10 topics, written for someone starting with zero PostgreSQL experience:

- 1 **Installation** — Windows installer walkthrough, `psql` vs `pgAdmin`, verifying setup.
- 2 **Database Management** — `psql` meta-commands (`\l`, `\c`, `\dt`, `\d`), creating your first database and tables from scratch.
- 3 **Roles & Permissions** — `CREATE ROLE/CREATE USER`, `GRANT/REVOKE`, least-privilege principle.
- 4 **Data Types** — PostgreSQL-specific types (`SERIAL`, `NUMERIC`, `UUID`, `TEXT`), why `NUMERIC` for money.
- 5 **Indexing** — B-Tree, Hash, GIN, GiST, BRIN — when to use each.
- 6 **JSONB** — semi-structured data inside a relational column, `->/->>/@>` operators.
- 7 **Arrays** — native array columns, 1-indexing, `unnest()`, when NOT to use them.
- 8 **Full Text Search** — `tsvector/tsquery`, why it beats `LIKE '%word%'`.
- 9 **Performance Tuning** — `EXPLAIN ANALYZE`, `VACUUM`, `ANALYZE`, `pg_stat_statements`.
- 10 **Backup & Restore** — `pg_dump/pg_restore`, plain vs custom format, testing restores.

You now have hands-on, practical PostgreSQL knowledge — from installing it on your own machine through advanced features — forming the direct database foundation for the Hibernate and Spring Data JPA sections ahead.

(END OF POSTGRESQL MASTER NOTES — ready to proceed to Hibernate whenever you say "Next Topic.")